SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1707

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 3 **Duration: 1 Hour**
Total Marks: 30

Annexure A

Dry Run Evaluation

Code of Conduct:

I **J O Joanna Divine / 192572003** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: J O Joanna Divine

Question 1 – Breadth-First Search (BFS)

Graph: A – (B, C) | B – (D, E) | C – (F, G)

Iteration	Queue (before expansion)	Node Expanded / Visited
1	A	A
2	B, C	B
3	C, D, E	C
4	D, E, F, G	D
5	E, F, G	E
6	F, G	F
7	G	G

1. BFS traversal order: A → B → C → D → E → F → G

2. Queue contents after each iteration:
After visiting A: [B, C] | After visiting B: [C, D, E] | After visiting C: [D, E, F, G] | After visiting D: [E, F, G] | After visiting E: [F, G] | After visiting F: [G] | After visiting G: [ ] (empty – search complete)

Question 2 – Depth-First Search (DFS)

Using the same graph, DFS starting from A (left child explored before right child):

Iteration	Stack (after push, top listed first)	Visited Node
1	B, C	A
2	D, E, C	B
3	E, C	D
4	C	E
5	F, G	C
6	G	F
7	(empty)	G

DFS traversal order: A → B → D → E → C → F → G

Question 3 – 8-Puzzle using BFS

Initial State	Goal State
1 3 6 5 0 2 4 7 8	1 2 3 4 5 6 7 8 0

Blank (0) moves swap positions with an adjacent tile (Up / Down / Left / Right). The dry run below traces the first 5 iterations of BFS from the initial state (states are written row-by-row, e.g. 136/502/478):

Iter	Current State (expanded)	Queue Before Expansion	Queue After Expansion
1	136/502/478	136/502/478	106/532/478, 136/572/408, 136/052/478, 136/520/478
2	106/532/478	106/532/478, 136/572/408, 136/052/478, 136/520/478	136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478
3	136/572/408	136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478	136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480
4	136/052/478	136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480	136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078
5	136/520/478	136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078	016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078, 130/526/478, 136/528/470

(BFS continues expanding level by level in the same manner — the queue keeps growing because the branching factor is 2–4 per state — until the goal state is dequeued.)

Answers

1. Node expanded in each iteration: Iteration 1 – 136/502/478 (initial state); Iteration 2 – 106/532/478; Iteration 3 – 136/572/408; Iteration 4 – 136/052/478; Iteration 5 – 136/520/478 (and so on, in first-in-first-out order of the queue).

2. Total states generated: Running BFS to completion, the search generates **838** child states in total, of which **500** are distinct (unvisited) states that get added to the queue before the goal state is reached.

3. Solution path (initial → goal), optimal length = 8 moves:

Step	State	Move applied
0	1 3 6 / 5 0 2 / 4 7 8	Start
1	1 3 6 / 5 2 0 / 4 7 8	Right
2	1 3 0 / 5 2 6 / 4 7 8	Up
3	1 0 3 / 5 2 6 / 4 7 8	Left
4	1 2 3 / 5 0 6 / 4 7 8	Down
5	1 2 3 / 0 5 6 / 4 7 8	Left
6	1 2 3 / 4 5 6 / 0 7 8	Down
7	1 2 3 / 4 5 6 / 7 0 8	Right
8	1 2 3 / 4 5 6 / 7 8 0	Right (Goal)

4. Why BFS always finds the shortest solution in an unweighted 8-puzzle: BFS explores the state space level by level (i.e., strictly in order of increasing number of moves from the start state). It fully expands every state reachable in k moves before it looks at any state reachable in k+1 moves. Because every move (edge) in the 8-puzzle has the same cost (1), the first time BFS dequeues the goal state, it is guaranteed to have been reached through the minimum possible number of moves – no shorter path could exist, since all shorter paths (fewer levels) were already exhausted without finding the goal. This makes BFS complete and optimal for unweighted search problems such as the 8-puzzle.